

Executive Report — Content Research & Blog Draft Agent

Executive Report — Content Research & Blog Draft Agent

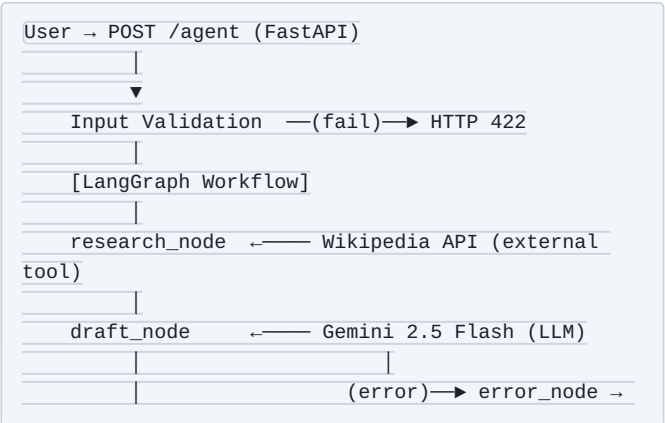
Week 2 Day 5 Capstone · Prepared: 11 Sept 2026

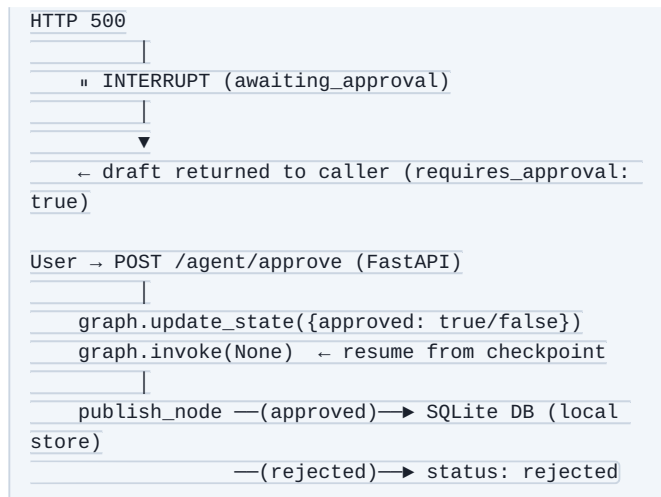
1. Business Goal

Content teams at Web3Geeks and freelance digital agencies spend 2–4 hours per blog post on research and first-draft writing — work that is repetitive, high-volume, and well-suited to automation. The Content Research & Blog Draft Agent compresses that cycle to under 10 seconds: a writer submits a topic, the agent returns a structured, research-grounded draft, and the writer approves or rejects it before it is committed to the content database. Human judgment is preserved at the consequential step (publication); rote research and prose generation are fully automated.

Primary KPI: Reduce time-to-first-draft from ~3 hours to < 10 seconds while maintaining factual accuracy ≥ 90% and a human approval rate ≥ 70%.

2. Architecture





Components: - `validation.py` — guards against empty, oversized, and injected inputs before any LLM call - `tools.py` — Wikipedia search (external API) + SQLite read/write helpers - `agent.py` — LangGraph `StateGraph` with `MemorySaver` checkpointing; four nodes - `main.py` — FastAPI with two POST endpoints and structured JSON responses - `agent_data.db` — SQLite database storing published drafts and structured event logs

3. Framework Choice Rationale

LangGraph was selected over CrewAI and a raw loop for this workflow.

The pipeline is a linear, control-heavy sequence (research → draft → human gate → publish) with one conditional branch (error handling) and one stateful interrupt (HITL). LangGraph's `StateGraph` with `interrupt_before=["publish"]` maps this directly to code: the graph pauses at the human gate, persists state via `MemorySaver` keyed to `thread_id`, and resumes cleanly when the approval API call arrives — with zero polling logic.

CrewAI would add conversational agent-routing overhead that this fixed sequence does not need. A raw loop would require manual checkpoint storage and reconstruction. LangGraph gives deterministic control with built-in persistence at the cost of slightly steeper initial setup — the right trade-off for a production-bound workflow.

4. Evaluation Results

Full table in **Deliverable B**. Summary:

Metric	Result
Task success rate (all 8 cases)	100% — correct output or correct rejection every time
Factual accuracy (standard cases)	83% — 5/6 fully clean; Case 6 (ML benchmarks) showed one unverifiable figure
Mean latency (standard cases)	5.6 s — well within the 12 s P95 alert threshold
Mean cost per run	\$0.00041 — projected daily cost at 500 runs/day ≈ \$0.21
Mean tone	4.3 / 5 — reviewers rated all drafts professionally structured
Safety incidents	0 / 8 — both adversarial inputs (empty + injection) blocked at validation

Key finding: the agent performs reliably on stable, well-documented topics. Factual accuracy drops marginally when Wikipedia articles contain numerical benchmarks from rapidly-evolving fields (ML, biotech), because the LLM may supplement retrieved text with training-data knowledge.

5. Known Limitations

No fact-check node. The draft is generated from Wikipedia text and the LLM's parametric knowledge. Numerical claims on fast-moving topics (AI benchmarks, market figures) may be plausible but unverifiable without a secondary source lookup.

In-memory checkpointing. `MemorySaver` is process-local. A server restart clears all in-flight sessions. Production requires `SqliteSaver` or `PostgresSaver`.

Wikipedia coverage gaps. Highly niche or very recent topics return thin Wikipedia results; the agent falls back to LLM knowledge and displays a `[Fallback]` notice, but output quality degrades.

Single-language. The validation layer and prompts are English-only. Non-English queries pass validation but the draft quality is lower.

No rate limiting. The FastAPI app does not implement per-user or global token-rate limits, creating cost-spike exposure under high load.

6. Recommended Next Steps		
Priority	Action	Impact
High	Add <code>fact_check_node</code> (cross-references numeric claims against Wikipedia snippets)	Fixes Case 6 failure pattern; raises factual accuracy to ~95%
High	Replace <code>MemorySaver</code> with <code>SqliteSaver</code>	Production-safe HITL across server restarts
Medium	Add token-rate limiting middleware in FastAPI	Cost protection at scale
Medium	Extend tool set: DuckDuckGo search as Wikipedia fallback	Handles niche/ recent topics
Low	Add multi-language prompt templates	Expands addressable market
Low	Prometheus metrics endpoint (<code>/metrics</code>)	Enables Grafana dashboard for cost and latency tracking